



Angular Avançado

Sessão 4

Resources, effects e o grafo de signals

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficamos na Sessão 3

Recapitulação rápida antes de continuar

Sub-componentes

- FlightCard: item **required** + `selected = model(false)`
- `(selectedChange) → updateBasket (spread)`
- `<ng-content>` **default + slot** `.footer`
- SimpleDelayStepper com `[(value)]="maxDelay"`

`computed()`

- Folha de cálculo: $B = f(A)$
- `flightRoute` **derivado do filter**
- **Lazy + memoizado + read-only**
- **Slicing** `from / to` = corte por igualdade
- `untracked()`: ler sem seguir

Já no repo

- `httpResource` com `defaultValue: []`
- `value · error · isLoading · status`
- `@let s = status() + @switch`
- **Signal Forms:** `form()` + `[formField]`

Hoje: os outros dois building blocks (resources e effects), as regras do jogo dos signals, e o grafo completo — depois, Boarding List.

Agenda de hoje

01 Resources

httpResource vs rxResource vs resource · params vs stream · snapshots

02 Effects

effect() · injection context · after*Render · inputs no constructor

03 Semântica dos signals

auto-tracking · untracking · glitch-free · igualdade e imutabilidade

04 O grafo completo

flightsWithDelays: computed + função pura + stepper

05 Exercício

Boarding List — níveis progressivos



01

Resources

Derivar de forma assíncrona: entra signal, sai signal

Três resources, uma API para o consumidor

	httpResource	rxResource	resource
Import	@angular/common/http	@angular/core/rxjs-interop	@angular/core
Entrada	request: () => { url, params }	params: () => {...}	params: () => {...}
Loader	(interno: HttpClient)	stream: (p) => Observable	loader: (p) => Promise
Quando	GET direto a uma Web API	Serviços com Observables · compor com RxJS	Raro — base dos outros dois
Cancelamento	automático	automático (unsubscribe)	via AbortSignal no loader

Comum a todos: value · error · isLoading · status · reload() · devolver undefined nos params desativa o resource · só a última resposta conta.

rxResource: quando já existe um Observable

```
flight-search.ts
import { rxResource } from '@angular/core/rxjs-interop';
import { Observable } from 'rxjs';

private readonly http = inject(HttpClient);

protected readonly flightsResource = rxResource({
  params: () => this.filter(), // o gatilho
  stream: ({ params: filter }) => this.find(filter.from, filter.to),
  defaultValue: [],
});

private find(from: string, to: string): Observable<Flight[]> {
  return this.http.get<Flight[]>(API_URL, { params: { from, to } });
}
```

`params:` (a chave) é a função que calcula o valor; `{ params }` (no `stream`) é o resultado dessa função. Dois nomes iguais, duas coisas diferentes.

rxjs-interop

- `rxResource` — `Observable` → `resource`
- `toSignal(obs$)` — `Observable` → `Signal`
- `toObservable(sig)` — `Signal` → `Observable`
- A ponte entre os dois mundos (módulo 5)

Anatomia do resource: gatilho e trabalhador



params — o gatilho

- Corre num **contexto reativo** (como um `computed`)
- Signals lidos aqui são seguidos
- Resultado diferente → novo carregamento
- `undefined` → `status() === 'idle'`, `stream` **não** corre

stream — o trabalhador

- Recebe o **resultado** de `params` em `{ params, abortSignal, previous }`
- Não é reativo: signals lidos aqui **não** são seguidos
- Devolve um `Observable`; o `resource` subscrive
- Novo `params` a meio → `unsubscribe` (pedido abortado) = `switchMap` grátis

Escrever "Lis" dispara um pedido que devolve `[]` — o `resource` funcionou; foi o backend que não encontrou nada. Diferença entre 'ainda não carregou' e 'não há resultados': `hasValue() + length`.

resource: a base, baseada em Promises

Promise-based

```
import { resource } from '@angular/core';

protected readonly flightsResource = resource({
  params: () => ({ from: this.filter().from, to: this.filter().to }),
  loader: ({ params, abortSignal }) =>
    this.findPromise(params.from, params.to, abortSignal),
  defaultValue: [],
});
```

- `httpResource` e `rxResource` são abstrações por cima deste
- Útil com `fetch()` nativo ou SDKs que devolvem Promises
- Se já tem um `Observable`, **não converta** — use `rxResource`

AbortSignal

- API do **browser** para cancelar operações assíncronas
- Nada a ver com Angular Signals
- Sem ele, o resource apenas **ignora** respostas antigas



02

Effects

Sair do grafo: a última milha até ao ecrã, à consola ou a uma lib externa

effect(): reagir com um side effect

```
flight-search.ts
import { effect } from '@angular/core';

constructor() {
  // corre 1x, e depois sempre que filter mudar
  effect(() => {
    const f = this.filter();
    console.log('From:', f.from, 'To:', f.to);
  });
}

// bom uso: algo que o data binding não consegue fazer
effect(() => {
  const error = this.error(); // unknown
  if (error instanceof HttpResponse) {
    this.snackBar.open(`Error ${error.status}: ${error.statusText}`,
      'OK');
  }
});
```

Quando corre

- 1× no arranque, **adiado** (não no constructor)
- Depois: sempre que um signal lido mudar
- Sem dependency array — **auto-tracking**

Para quê

- "Última milha": canvas, toast, log, storage, libs sem signals
- **Não** para escrever noutros signals nem para lógica de negócio

Injection context: onde effect() pode ser criado

onde existe, onde não existe

```
export class FlightSearch {
  private readonly http = inject(HttpClient);           // ✅ field initializer
  private readonly injector = inject(Injector);         // ✅ guardar
  protected readonly flightsResource = rxResource({ ... }); // ✅

  constructor() {
    effect(() => { ... });                             // ✅ constructor
  }

  ngOnInit() {
    effect(() => { ... });                               // ❌ NG0203
    effect(() => { ... }, { injector: this.injector }); // ✅ escape hatch
  }
}
```

O crachá

- Enquanto o Angular **constrói** a classe, sabe quem está a pedir → `inject()` funciona
- Constructor + field initializers = com crachá
- `ngOnInit`, `handlers`, `setTimeout`, `subscribe` = sem crachá
- Precisam dele: `inject`, `effect`, `afterNextRender`, `toSignal`

Os cabos instalam-se quando a casa se constrói. Field initializers correm dentro do constructor, pela ordem em que aparecem — por isso `flightsResource` pode ler `this.filter` declarado acima.

Effects que precisam do DOM pronto

Função	Dispara quando	Reativo?	≈ React	Uso típico
<code>effect()</code>	signal lido muda (antes do render)	sim	<code>useEffect(fn, [a, b])*</code>	log, toast, storage
<code>afterRenderEffect()</code>	signal lido muda, depois do render	sim	—	medir/alterar DOM em função de estado
<code>afterNextRender()</code>	1× após o próximo render	não	<code>useEffect(fn, [])</code>	iniciar lib de charts, focus
<code>afterEveryRender()</code>	após cada render (mesmo sem signals)	não	<code>useEffect(fn)</code>	sincronizar lib externa

- * com deps **descobertas**, não declaradas — o `effect` dispara pelo grafo de signals, não pelo render
- `afterEveryRender` com um signal lá dentro é quase sempre engano: se depende de estado, é `effect`
- `after*Render` não correm em SSR — é a razão de existirem em vez de `ngAfterViewInit`

Signals no ciclo de vida: inputs e o constructor

```
flight-card.ts
export class FlightCard {
  readonly item = input.required<Flight>();

  // ✅ computed é lazy: só lê item() quando alguém lê flightRoute()
  protected readonly flightRoute = computed(
    () => `${this.item().from} → ${this.item().to}`,
  );

  constructor() {
    // console.log(this.item()); ❌ NG0950: input não inicializado
    effect(() => console.log('Item:', this.item())); // ✅ adiado
  }
}
```

Regra

- Constructor = instânciação; o binding inicial vem **depois**
- `effect` no constructor é adiado até à inicialização
- `computed` só lê quando é lido
- Adeus `ngOnChanges` para derivar de inputs

Tempo: `debounced()` (Angular 22)

Opcional

```
debounce de um signal
import { debounced, effect, signal } from '@angular/core';

const filter = signal('');
const debouncedFilter = debounced(filter, 300); // → Resource

effect(() => console.log(debouncedFilter.value()));
// filter muda a cada tecla; value() só após 300 ms estável
// debouncedFilter.status() === 'loading' na janela
```

Porquê um Resource?

- Signals **não têm tempo**; resources sim
- `value lag` + `status` de espera

Armadilha: `debounced()` devolve um Resource read-only — não pode substituir o signal escrevível que o form usa. Manter os dois: o original para o form, o debounced para a pesquisa. Para debounce por campo, Signal Forms traz `debounce()` (módulo 7).



03

Semântica dos signals

As quatro regras que explicam quase todos os 'porque é que não atualizou?'

Auto-tracking: o que é seguido, e onde

tracking atravessa chamadas

```
constructor() {  
  effect(() => {  
    this.logCriteria(); // segue filter, sem o mostrar  
  });  
}  
  
private logCriteria(): void {  
  const f = this.filter(); // tracked, indiretamente  
  console.log(f.from, '→', f.to);  
}
```

lógica de negócio num effect

```
effect(() => {  
  const criteria = this.criteria();  
  this.service.executeLogic(criteria); // ✗  
});  
  
// se executeLogic lê isLoading() ou userId()...  
// ...o effect re-corre quando ELES mudam.  
// e não há tratamento de race conditions.
```

Contexto reativo existe em

Templates, effects, computed e params de resources. Ler `x()` duas vezes = uma dependência.

Consequência

Effects para rendering: ótimo. Effects para negócio: re-execuições surpresa + sem race handling.

Explicit effects e untracking condicional

"explicit effect" — o mais perto de um dependency array

```
effect(() => {
  const filter = this.filter(); // ✅ só isto dispara
  untracked(() =>
    this.logSearch(filter.from, filter.to));
});

private logSearch(from: string, to: string) {
  // flights() aqui dentro NÃO é seguido
  console.log(`${from} → ${to}`,
    this.flights().length); // ⚠️ pesquisa anterior
}
```

dependências dinâmicas

```
effect(() => {
  if (isDelayed()) {
    console.log(delay()); // só seguido se isDelayed
  }
});

effect(() => {
  const isDelayedValue = isDelayed();
  const delayValue = delay(); // ler no início
  if (isDelayedValue) console.log(delayValue);
});
```

- `untracked` **esconde** a dependência de `flights`, não a elimina — o log mostra a pesquisa anterior
- Um signal que **deixa de ser lido** numa execução deixa de ser seguido — se deve disparar sempre, leia-o no início (numa `const`)
- Vale para templates também: um `@if` fechado deixa de seguir o que está lá dentro

Glitch-free: nunca ver estados intermédios

5 updates, 1 execução

```
constructor() {  
  effect(() => {  
    const f = this.filter();  
    console.log('from:', f.from, 'to:', f.to);  
  });  
  
  setTimeout(() => {  
    this.filter.update(f => ({ ...f, from: 'Paris' }));  
    this.filter.update(f => ({ ...f, from: 'Frankfurt' }));  
    this.filter.update(f => ({ ...f, from: 'New York' }));  
    this.filter.update(f => ({ ...f, to: 'Berlin' }));  
    this.filter.update(f => ({ ...f, to: 'London' }));  
  }, 2000);  
}  
// consola: 'from: New York to: London' — UMA vez
```

Como

- set só **marca** consumidores como sujeitos
- Re-execução agendada lê os valores **finais**

Consequência

- Signals **não** representam eventos nem streams no tempo
- Mensagens rápidas perdem-se → RxJS

Igualdade e imutabilidade

```
Object.is e spread
const count = signal(0);
count.set(0); count.set(0); // Object.is igual → nada acontece

// objetos: compara REFERÊNCIA
flight.update(f => ({ ...f, date: newDate, delayed: true }));
// shallow copy + override → nova instância → detetada

// ❌ mutação in-place: mesma referência, ninguém repara
flight().delayed = true;
basket.update(b => { b[id] = true; return b; }); // ❌ idem
```

E o OnPush?

- No @for, cada flight é comparado por ===
- Objeto novo → só **esse** FlightCard é revisitado
- Imutabilidade é o que torna OnPush barato

Regra curta: signals não observam objetos por dentro; observam trocas de valor. Ao atualizar objetos ou arrays, crie sempre novas instâncias. Custom equality (2.º arg. de signal()) existe, mas raramente compensa.

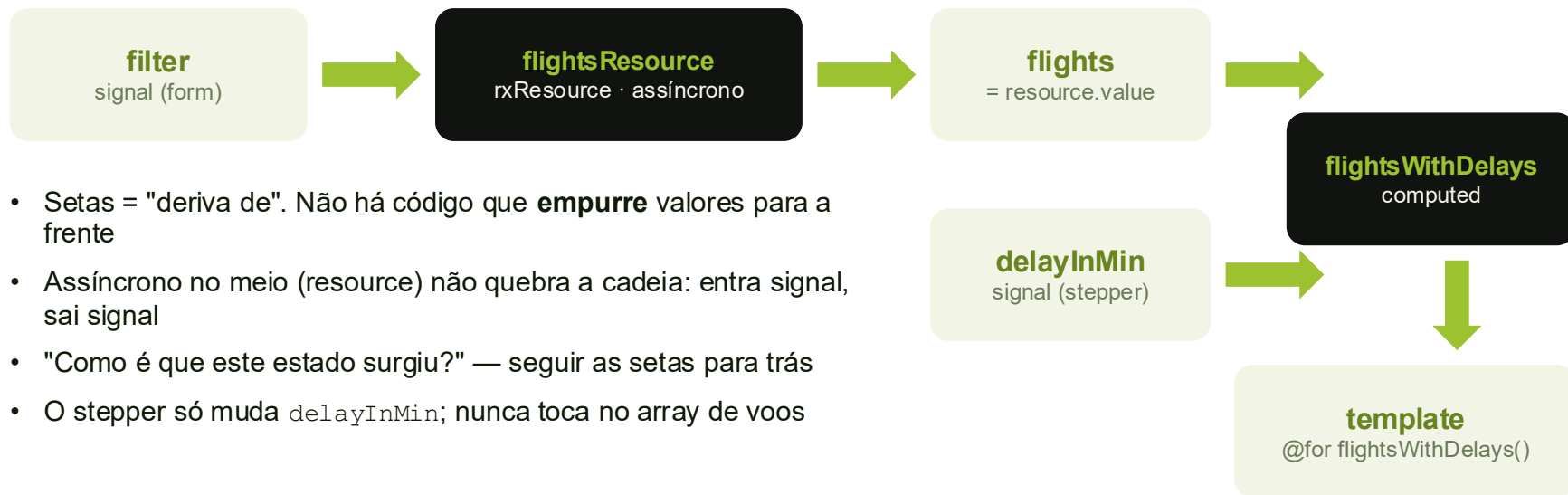


04

O grafo completo

Um valor só de cliente (delay) que entra no fluxo sem tocar nos dados

■ ■ ■ Pensar no grafo de signals



■ Implementar o fluxo: computed + função pura

```
flight-search.ts / .html

protected readonly flights = this.flightsResource.value;
protected readonly delayInMin = signal(0); // era maxDelay

protected readonly flightsWithDelays = computed(() =>
  toFlightsWithDelays(this.flights(), this.delayInMin()),
);

<!-- o stepper que já existe passa a alimentar o grafo -->
<app-simple-delay-stepper [(value)]="delayInMin" />
```

```
helper puro

// fora da classe: pura, sem signals
function toFlightsWithDelays(
  flights: Flight[], delay: number): Flight[] {
  if (flights.length === 0) return [];
  const MIN = 60_000;
  const [first, ...rest] = flights;
  const t = new Date(first.date).getTime();
  const newDate = new Date(t + delay * MIN);
  const newFirst = {
    ...first, date: newDate.toISOString(),
    delayed: delay > 0, // derivado
  };
  return [newFirst, ...rest]; // novo array
}
```

```
@for (flight of flightsWithDelays(); track flight.id) { <app-flight-card [item]="flight" ... /> }
Date: {{ item().date | date: 'dd.MM.yyyy HH:mm' }} <!-- flight-card.html: sem a hora, +15 min não se vê -->
```

Funções puras fora da classe: sem acesso a `this`, e obrigam a ler os `signals` dentro do `computed` — as dependências ficam visíveis à primeira.

Relembrar: o que acabámos de escrever

O fluxo reativo no Skyboard

O que ficou no código

- `rxResource (params / stream)` no lugar do `httpResource` — `template` intacto
- `effect` no constructor: log do `filter` + erro (snackbar ou consola)
- `delayInMin` + `flightsWithDelays` (computed + helper puro; delayed derivado)
- Stepper ligado a `delayInMin`; `@for` sobre `flightsWithDelays ()`; card com a hora

Para explorar

- Provar o glitch-free: 3 `update` seguidos, 1 `log`
- Pôr um `afterEveryRender` com `console.log` e clicar no stepper
- Voltar ao `httpResource` sem mudar o `template`

Pergunta para discutir: onde é que, no vosso código atual, um `ngOnChanges` ou um `subscribe` podia ser um `computed`?



05

Exercício — Boarding List

Consolidação: signals + computed · níveis progressivos · sem HTTP

Exercício — Boarding List

Níveis 1 e 2 em sessão · 3 e 4 para casa · snippets: bloco EX

```
setup
ng g c domains/ticketing/feature-
boarding/boarding-list

export interface Passenger {
  id: number;
  name: string;
  boarded: boolean;
}
```

Nível 1 — Lista (≈12 min)

- `passengers = signal<Passenger[]>([...])` com 3 iniciais
- `newName = signal('') + form() + [formField]`
- `add(): update` com array novo; depois `newName.set('')`
- `toggle(id): map + spread` → objeto novo
- `@for ... track p.id + @empty + checkbox`

Nível 2 — Lote e contagem (≈10 min)

- "Embarcar todos" e "Reset" com `update + map`
- `boardedCount` e `total` com `computed()`
- `allBoarded = computed(...)` encadeado → `@if "Porta fechada"`

Perguntas para discutir

- Porque é que `passengers().push(novo)` não atualiza o ecrã?
- Porque é que `p.boarded = !p.boarded` no `map` é má ideia?
- `boardedCount` recalcula ao escrever em `newName`? Porque não?

Exercício — Boarding List (cont.)

Níveis 3 e 4 · o que sobrar fica para casa

Nível 3 — Filtro (≈10 min)

- `filter = signal<'all' | 'boarded' | 'pending'>('all')`
- 3 botões com `filter.set(...)` e `[class.active]`
- `visible = computed(...)` que filtra por `filter()`
- `@for` passa a iterar `visible()`
- `@switch` para o título, com `@default never`

```
@let f = filter();           // sem isto, @default never não compila
@switch (f) {
  @case ('all') { <h2>Todos</h2> }
  @case ('boarded') { <h2>Embarcados</h2> }
  @case ('pending') { <h2>Por embarcar</h2> }
  @default never;
}
```

Nível 4 — Sub-componente (bônus)

- `ng g c domains/ticketing/ui/passenger-row`
- `passenger = input.required<Passenger>()`
- `boardedChange = output<boolean>()`
- `<app-passenger-row [passenger]="p" (boardedChange)="toggle(p.id)" />`
- Refatorar para `model(false)` — dá para `[(boarded)]`?

Pergunta final: o que acontece se adicionarem 'crew' ao union type sem `@case`? Experimentem — e guardem a resposta para a próxima sessão.

Resumo da sessão

Resources

- `httpResource` · `rxResource` · `resource`
- `params` = gatilho reativo
- `stream` = trabalhador, não reativo
- `undefined` desativa · `race` conditions grátis

Effects

- Última milha: log, toast, DOM, libs
- Injection context: constructor / field init
- `after*Render` para o DOM
- Inputs: nunca no constructor

Semântica

- Auto-tracking · `untracked`
- Glitch-free → não é para eventos
- `Object.is` → imutabilidade
- Pensar no grafo

Para casa: Boarding List (níveis que faltarem). Próxima sessão: Router — navegação, rotas parametrizadas, lazy loading.
Antes: git pull → tag sessao-04.